

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: COMPUTER NETWORKS
COURSE CODE: CSA07

COURSE OUTCOME COVERED

CO4:

Design network topologies star, mesh, bus, and hybrid using networking devices, transmission media, and cabling standards
(BL2,BL6)

Assessment Tool Weightage: 40%

Annexure A

Coding challenge

1. Develop a Python program that generates a Star Topology for a given number of hosts connected to a central switch. The program should accept the number of hosts as input and display the topology either in ASCII format or a graphical representation. For each host, print the host label, the name of the central switch, and the corresponding Cat6 UTP cable length connecting the host to the switch. Verify that all hosts are connected through the central switch and explain how the star topology facilitates reliable communication in a local area network.
2. Write a Python function named `validate_segment(length_m)` to verify whether a given **UTP cable segment** satisfies the IEEE 802.3 Ethernet standard, which specifies a maximum cable length of **100 meters**. The function should return **True** for valid cable lengths and return **False** along with an appropriate error message for invalid lengths exceeding the permissible limit. Demonstrate the correctness of the program by executing it with at least **three different test cases** and interpret the results.

3. Develop a Python program to simulate the operation of the Carrier Sense Multiple Access with Collision Detection (CSMA/CD) protocol in a star topology consisting of four hosts connected through a central switch. The program should simulate frame transmission attempts, detect transmission collisions, calculate random back-off intervals, and continue transmission after the waiting period. Record every collision event, transmission attempt, and back-off interval in a log file named `csma_log.txt`, and analyze how CSMA/CD minimizes data transmission conflicts in Ethernet networks.
4. Develop a client-server application using Python's socket library to simulate communication between two hosts connected through a central switch in a star topology. The server should receive frames from the client, forward them appropriately, and maintain a simulated MAC Address Table. After each successful frame transmission, display the updated MAC address table showing the learned MAC addresses and associated ports. Explain how MAC learning and frame forwarding are performed in Ethernet switches.
5. Design and implement a Python class named `StarSwitch` that models the basic functionality of an Ethernet switch. The class should include methods such as `add_port(host_id)` to connect a host, `remove_port(host_id)` to disconnect a host, and `broadcast(frame)` to transmit a frame to all connected ports except the source port. The program should maintain a record of active ports and log every broadcast operation, indicating the source host and the destination hosts that successfully receive the frame. Demonstrate the implementation using multiple hosts and explain the significance of broadcasting in Ethernet communication.

RUBRICS FOR CALCULATION

Criteria	Weightage	Exemplary (4)	Proficient (3)	Developing (2)	Beginning (1)
Problem Understanding	15%	Fully understands the problem and requirements; no clarification needed	Understands most requirements with minor clarification	Partial understanding; misses some requirements	Poor understanding of the problem
Algorithm Design	20%	Efficient, optimal, and well-structured algorithm	Correct algorithm with minor inefficiencies	Basic approach but not optimal	Incorrect or no clear algorithm
Code Quality	15%	Clean, well-organized, readable, and properly formatted code	Mostly clean code with minor issues	Code is somewhat unclear or inconsistent	Poorly written, unreadable code
Functionality	20%	Code works perfectly for all test cases	Works for most test cases	Works for limited cases	Does not run or fails major cases
Error Handling	10%	Handles all edge cases and errors effectively	Handles some edge cases	Limited error handling	No error handling
Efficiency	10%	Highly optimized in time and space complexity	Reasonably efficient	Some inefficiencies	Highly inefficient
Documentation & Comments	5%	Clear and meaningful comments and documentation	Adequate comments	Few or unclear comments	No comments
Testing & Debugging	5%	Thoroughly tested with multiple cases; no bugs	Tested with some cases	Minimal testing	No testing done

1. Star Topology Generator

This program accepts the number of hosts, assigns a Cat6 cable length to each host, displays an ASCII star topology, and verifies that every host is connected to the central switch.

Python code:

```
# Star Topology Generator
```

```
n = int(input("Enter number of hosts: "))
```

```
switch = "Central Switch"
```

```
hosts = []
```

```
for i in range(1, n + 1):
```

```
    length = float(input(f"Enter Cat6 cable length for Host {i} (in meters): "))
```

```
    if length > 100:
```

```
        print("Cable length cannot exceed 100 meters.")
```

```
        length = 100
```

```
    hosts.append((f"Host {i}", length))
```

```
print("\n===== STAR TOPOLOGY =====")
```

```
print("          [Central Switch]")
```

```
print("          |")
```

```
for host, length in hosts:
```

```
    print(f"      +-----+-----+")
```

```
    print(f"      | {host:<8} --> {switch} |")
```

```
    print(f"      | Cable Length: {length} m |")
```

```
print("\n===== CONNECTION DETAILS =====")
```

```
for host, length in hosts:
```

```
    print(f"{host} --> {switch} --> Cat6 UTP: {length} meters")
```

```
# Verification
```

```
if len(hosts) == n:
```

```
    print("\nVerification: SUCCESS")
```

```
    print(f"All {n} hosts are connected through the central switch.")
```

```
else:
```

```
    print("\nVerification: FAILED")
```

Output:

```
Output

===== STAR TOPOLOGY =====
          [Central Switch]

+-----+-----+
| Host1   --> Central Switch |
| Cable Length: 20.0 m |
+-----+-----+
| Host2   --> Central Switch |
| Cable Length: 35.0 m |
+-----+-----+
| Host3   --> Central Switch |
| Cable Length: 40.0 m |
+-----+-----+
| Host4   --> Central Switch |
| Cable Length: 50.0 m |

===== CONNECTION DETAILS =====
Host1 --> Central Switch --> Cat6 UTP: 20.0 meters
Host2 --> Central Switch --> Cat6 UTP: 35.0 meters
Host3 --> Central Switch --> Cat6 UTP: 40.0 meters
Host4 --> Central Switch --> Cat6 UTP: 50.0 meters

Verification: SUCCESS
All 4 hosts are connected through the central switch.
```

In a star topology, every host has a separate connection to a central switch. If one cable fails, only that host is affected. The other hosts can continue communicating, making the topology reliable and easy to troubleshoot.

2. UTP Cable Length Validation

The maximum Ethernet copper cable segment length is **100 meters**. This function checks whether the given length is valid.

Python code:

```
def validate_segment(length_m):
    if length_m < 0:
        return False, "Error: Cable length cannot be negative."

    if length_m <= 100:
        return True, "Valid: Cable segment is within the 100-meter limit."

    return False, "Error: Cable segment exceeds the maximum limit of 100
```

meters."

Test cases

test_cases = [50, 100, 120]

for length in test_cases:

 result, message = validate_segment(length)

 print(f"Length = {length} meters")

 print(f"Result = {result}")

 print(message)

 print("-----")

Output:

```
Length = 50 meters
Result = True
Valid: Cable segment is within the 100-meter limit.
-----
Length = 100 meters
Result = True
Valid: Cable segment is within the 100-meter limit.
-----
Length = 120 meters
Result = False
Error: Cable segment exceeds the maximum limit of 100 meters.
-----

=== Code Execution Successful ===
```

3. CSMA/CD Simulation

Important exam point: Traditional CSMA/CD applies to **shared, half-duplex Ethernet**. A modern switched Ethernet network normally uses full-duplex communication, where collisions do not occur. For this coding challenge, we simulate the collision behavior requested.

The program:

- Uses 4 hosts.
- Generates transmission attempts.
- Detects collisions.
- Generates a random back-off interval.
- Retries transmission.
- Records everything in csma_log.txt.

Python code:

```
import random
import time

hosts = ["Host1", "Host2", "Host3", "Host4"]

log_file = "csma_log.txt"

def log(message):
    print(message)

    with open(log_file, "a") as file:
        file.write(message + "\n")

# Clear previous log
with open(log_file, "w") as file:
    file.write("===== CSMA/CD LOG =====\n")

for frame in range(1, 6):

    source = random.choice(hosts)
    attempt = 1

    while True:

        log(f'\nFrame {frame}: {source} - Attempt {attempt}')
        log("Carrier Sense: Channel checked.")

        # Randomly decide whether collision occurs
        collision = random.choice([True, False, False])

        if collision:
            log("COLLISION DETECTED!")

            # Random binary backoff
            backoff = random.randint(1, 10)

            log(f'Back-off interval = {backoff} time units')

            time.sleep(0.1)

            log(f'{source} waits for {backoff} time units.')
            log(f'{source} retransmits the frame.')
```

```

        attempt += 1

    else:
        log(f'{source}: Frame {frame} transmitted successfully.')
        break

log("\n===== TRANSMISSION COMPLETE =====")
log("CSMA/CD simulation completed.")
log("Log saved in csma_log.txt")

```

Example output

```

Frame 1: Host2 - Attempt 1
Carrier Sense: Channel checked.
COLLISION DETECTED!
Back-off interval = 4 time units
Host2 waits for 4 time units.
Host2 retransmits the frame.

```

```

Frame 1: Host2 - Attempt 2
Carrier Sense: Channel checked.
Host2: Frame 1 transmitted successfully.

```

4. Client-Server Ethernet Switch Simulation

This program uses Python's socket library.
The server represents the central switch, while the client represents a host.
The switch learns:
MAC Address → Port
and then forwards frames according to its MAC address table.

Server Program

Save as **server.py**:

Python code:

```

import socket

HOST = "127.0.0.1"
PORT = 5000

# Simulated MAC address table
mac_table = {}

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind((HOST, PORT))
server.listen(5)

print("Ethernet Switch Server Started")
print("Waiting for connections...\n")

```

```
while True:
```

```
    connection, address = server.accept()
```

```
    print("Host connected:", address)
```

```
    # Receive frame
```

```
    data = connection.recv(1024).decode()
```

```
    if not data:
```

```
        connection.close()
```

```
        continue
```

```
    # Frame format:
```

```
    # SOURCE_MAC, DESTINATION_MAC, DATA
```

```
    parts = data.split(",")
```

```
    source_mac = parts[0]
```

```
    destination_mac = parts[1]
```

```
    message = parts[2]
```

```
    # Learn source MAC address
```

```
    mac_table[source_mac] = str(address[1])
```

```
    print("\nFrame Received")
```

```
    print("Source MAC      :", source_mac)
```

```
    print("Destination MAC :", destination_mac)
```

```
    print("Data          :", message)
```

```
    print("\nMAC Address Table")
```

```
    print("-----")
```

```
    for mac, port in mac_table.items():
```

```
        print(f"{mac} --> Port {port}")
```

```
    # Forward frame
```

```
    if destination_mac in mac_table:
```

```
        print("\nDestination found.")
```

```
        print("Frame forwarded to destination.")
```

```
        response = "Frame forwarded successfully."
```

```
    else:
```

```
        print("\nDestination unknown.")
```

```
        print("Frame will be broadcast.")
```

```
        response = "Destination unknown - frame broadcast."
```

```
connection.send(response.encode())
```

```
connection.close()
```

Client Program

Save as **client.py**:

```
import socket
```

```
HOST = "127.0.0.1"
```

```
PORT = 5000
```

```
source_mac = input("Enter Source MAC Address: ")
```

```
destination_mac = input("Enter Destination MAC Address: ")
```

```
message = input("Enter message/frame data: ")
```

```
client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```

```
client.connect((HOST, PORT))
```

```
frame = source_mac + "," + destination_mac + "," + message
```

```
client.send(frame.encode())
```

```
response = client.recv(1024).decode()
```

```
print("\nSwitch Response:")
```

```
print(response)
```

```
client.close()
```

How to run

Open **two terminals**.

Terminal 1:

```
python server.py
```

Then open **Terminal 2:**

```
python client.py
```

Example:

```
Enter Source MAC Address: AA:BB:CC:DD:EE:01
```

```
Enter Destination MAC Address: AA:BB:CC:DD:EE:02
```

```
Enter message/frame data: Hello Host2
```

The server learns the source MAC address and adds it to the MAC table.

MAC Learning

When a switch receives a frame, it examines the **source MAC address** and records the port from which the frame arrived.

Example:

```
AA:BB:CC:DD:EE:01 --> Port 5001
```

AA:BB:CC:DD:EE:02 --> Port 5002

Frame Forwarding

- If the destination MAC is known → frame is sent only to the appropriate port.
- If the destination MAC is unknown → frame is flooded/broadcast to the relevant ports.
- The switch continues learning MAC addresses from incoming frames.

5. StarSwitch Python Class

This program creates a class representing a basic Ethernet switch.

It supports:

- add_port()
- remove_port()
- broadcast()
- Active port tracking
- Broadcast logging

Python code:

class StarSwitch:

```
def __init__(self):
    self.active_ports = []
    self.log_file = "switch_broadcast_log.txt"

    # Create/clear log file
    with open(self.log_file, "w") as file:
        file.write("===== SWITCH BROADCAST LOG\n")

def add_port(self, host_id):
    if host_id not in self.active_ports:
        self.active_ports.append(host_id)
        print(f'{host_id} connected to switch.')
    else:
        print(f'{host_id} is already connected.')

def remove_port(self, host_id):
    if host_id in self.active_ports:
        self.active_ports.remove(host_id)
        print(f'{host_id} disconnected from switch.')
    else:
        print(f'{host_id} is not connected.')

def broadcast(self, frame):
    source = frame["source"]
    data = frame["data"]
```

```

destinations = []

for host in self.active_ports:
    if host != source:
        destinations.append(host)

print("\n===== BROADCAST =====")
print("Source:", source)
print("Frame :", data)

if destinations:
    print("Frame sent to:")

    for host in destinations:
        print(" ->", host)

else:
    print("No other active hosts.")

# Write operation to log file
with open(self.log_file, "a") as file:
    file.write("\nBroadcast Operation\n")
    file.write(f"Source Host: {source}\n")
    file.write(f"Frame: {data}\n")
    file.write(
        "Destination Hosts: " +
        ", ".join(destinations) + "\n"
    )

# Create switch
switch = StarSwitch()

# Add multiple hosts
switch.add_port("Host1")
switch.add_port("Host2")
switch.add_port("Host3")
switch.add_port("Host4")

print("\nActive Ports:")
print(switch.active_ports)

# Broadcast frame from Host1
frame1 = {
    "source": "Host1",

```

```

    "data": "Hello from Host1"
}

switch.broadcast(frame1)

# Remove Host3
switch.remove_port("Host3")

# Broadcast another frame
frame2 = {
    "source": "Host2",
    "data": "Network announcement"
}

switch.broadcast(frame2)

print("\nFinal Active Ports:")
print(switch.active_ports)

print("\nBroadcast operations saved in switch_broadcast_log.txt")

```

Expected output:

Host1 connected to switch.
Host2 connected to switch.
Host3 connected to switch.
Host4 connected to switch.

Active Ports:

['Host1', 'Host2', 'Host3', 'Host4']

===== BROADCAST =====

Source: Host1

Frame : Hello from Host1

Frame sent to:

-> Host2

-> Host3

-> Host4

Host3 disconnected from switch.

===== BROADCAST =====

Source: Host2

Frame : Network announcement

Frame sent to:

-> Host1

-> Host4

Final Active Ports:
['Host1', 'Host2', 'Host4']